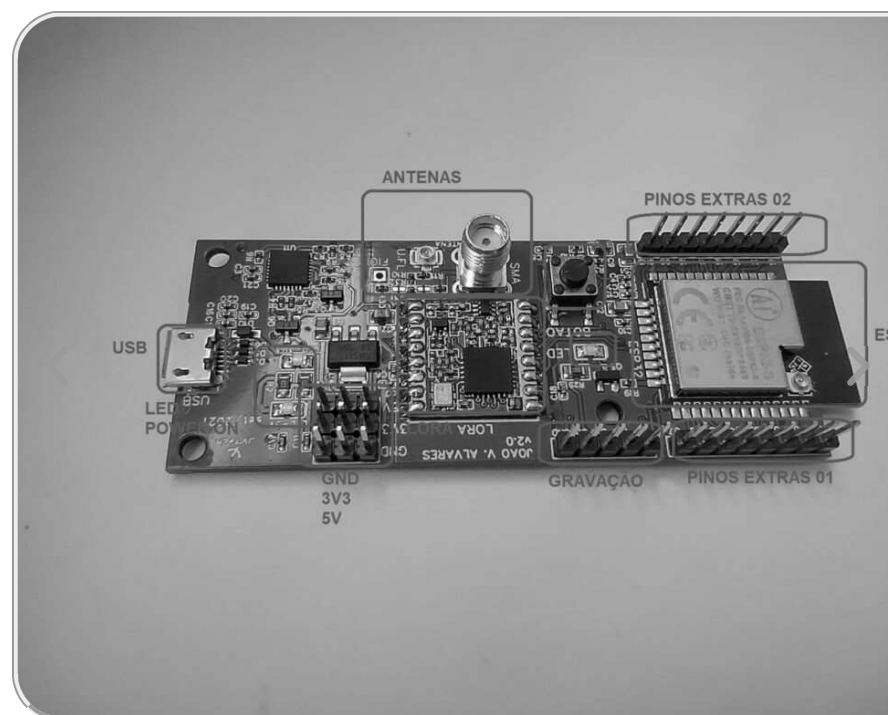


## Comparando as versões v2.0 e v4.0 da Placa ESP32 LoRa JVTech

### ▲ Apresentação do ESP32 e do LoRa

Entre os diversos módulos de microprocessamento o ESP32 é um dos mais populares. Por permitir a elaboração de circuitos que envolvam conexões por meio da internet, *bluetooth* ou outros meios. Os sistemas que precisam de acesso remoto e que fazem parte da internet das coisas (IoT) normalmente possuem algum módulo similar a esse. Sua programação é feita por meio de um cabo micro USB que pode ser conectado a um computador ou por outras maneiras.

Quanto a tecnologia LoRa, que significa longo alcance (*Long Range* no inglês original), é possível fazer a recepção e transmissão de dados, assim como um monitoramento ou acesso remoto de um circuito. Isso permite uma comunicação sem fio que é concretizada a partir de rádio frequência.



Dessa forma, ao integrarmos essas duas tecnologias podemos criar um circuito que é processado por meio do ESP32 e que faz parte da internet das coisas, seja por meio do Wi-Fi ou da rádio frequência. Essa placa ESP32 LoRa JVTech permite fazer isso sem a necessidade de componentes adicionais para relacionar o hardware do ESP32 com o LoRa.

#### Usos

O monitoramento de sensores de temperatura, luminosidade, movimento, entre outros, até a transmissão e recepção de dados a certa distância e circuitos diversificados são possibilidades que podem ser concretizadas com esse módulo. Além disso, a possibilidade de acesso remoto com interfaces em computadores, aplicativos para aparelhos celulares, ou até outros módulos que possuam a mesma possibilidade de transmissão e recepção. Há uma vasta gama de possibilidades de uso que variam das necessidades do projeto em questão.

#### ▲ Especificações

| ESP32 |                              |
|-------|------------------------------|
| CPU   | Xtensa® Dual-Core 32-bit LX6 |
| ROM   | 448 KBytes                   |
| RAM   | 520 Kbytes                   |

## Nátali Vieira Sardi



Estudante de Engenharia Elétrica, com experiência em eletrônica, programação, escrita e fluente em inglês. Curiosa sobre o modo de funcionamento das coisas, gosto de aprender sobre componentes eletrônicos, diversas linguagens de programação, a língua alemã e métodos de passar esses conhecimentos a outros.



**@natii.vs**

|                        |                                                                     |
|------------------------|---------------------------------------------------------------------|
| <b>FLASH</b>           | 4 MB                                                                |
| <b>Clock máximo</b>    | 240MHz                                                              |
| <b>Wireless padrão</b> | 802.11 b/g/n                                                        |
| <b>Conexão Wi-Fi</b>   | 2.4Ghz (máximo de 150 Mbps)                                         |
| <b>Antena</b>          | Embutida                                                            |
| <b>Conexão</b>         | Wi-Fi Direct (P2P), P2P Discovery, P2P Group Owner mode e P2P Power |

|                           |               |
|---------------------------|---------------|
| <b>LoRa</b>               |               |
| <b>Controlador</b>        | SX1278        |
| <b>Tensão de operação</b> | 2,2 V e 3,6 V |

|                           |               |
|---------------------------|---------------|
| Potência máxima           | 20dBm         |
| Sensibilidade do receptor | -146dBm       |
| Conector para antena      | SMA e Pigtail |
| Corrente de emissão       | 130mA         |
| Corrente de recepção      | 13.5mA        |
| <i>Sleep current</i>      | 2.0uA         |
| Interface de comunicação  | SPI           |
| Frequência de operação    | 915MHZ        |
| Temperatura de operação   | -30 ~ 85°C    |

|                          | Placa ESP32 LoRa<br>v4.0 | Placa ESP32 LoRa<br>v2.0 |
|--------------------------|--------------------------|--------------------------|
| Cor da placa             | Preto                    | Verde                    |
| Distância entre<br>pinos | 2,54                     | 2,54                     |
| Dimensões                | 68mm x 32mm x<br>10mm    | 80mm x 32mm x<br>10mm    |

|                              |               |               |
|------------------------------|---------------|---------------|
| <b>Modos de operação</b>     | STA/AP/STA+AP | STA/AP/STA+AP |
| <b>Bluetooth</b>             | BLE 4.2       | BLE 4.2       |
| <b>Portas GPIO</b>           | 11            | 11            |
| <b>Tensão de operação</b>    | 4,5 ~ 9V      | 4,5 ~ 9V      |
| <b>Taxa de transferência</b> | 110-460800bps | 110-460800bps |

## ▲ Hardware da placa

As duas versões da Placa ESP32 LoRa possuem basicamente os pinos que são análogos aos presentes no ESP32. Ambas as placas possuem oito partes principais indicadas nas figuras 1 e 2 a seguir, essas partes correspondem aos pinos, conector para antena, alimentação, entre outros. Na tabela a seguir as placas e suas áreas são comparadas:

| <b>Índice</b> | <b>Placa ESP32 LoRa v4.0</b> | <b>Placa ESP32 LoRa v2.0</b> |
|---------------|------------------------------|------------------------------|
| 1             | Pinos de uso geral (GPIO)    | Pinos de uso geral (GPIO)    |
|               |                              |                              |

|   |                           |                           |
|---|---------------------------|---------------------------|
| 2 | Pinos de gravação externa | Pinos de gravação externa |
| 3 | Pinos de uso geral (GPIO) | Pinos de uso geral (GPIO) |
| 4 | Alimentação da placa      | Alimentação da placa      |
| 5 | Conectores para antena    | Conectores para antena    |
| 6 | Botão                     | Botão                     |
| 7 | LED de uso geral          | LED de uso geral          |
| 8 | Conector micro-USB tipo B | Conector micro-USB tipo B |

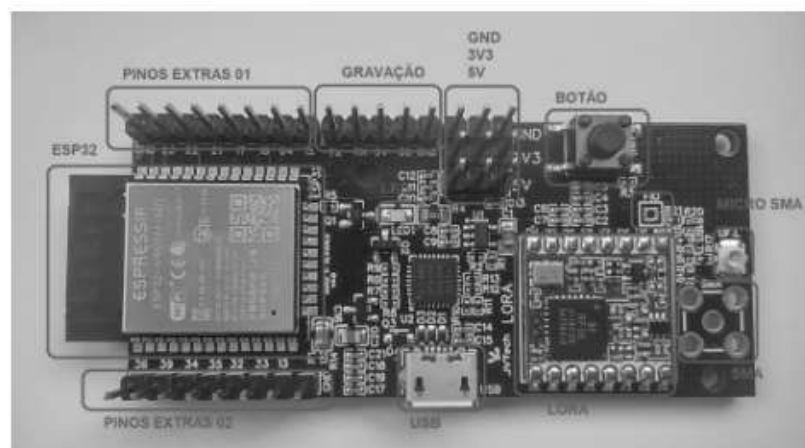


FIGURA 1 – Placa ESP32 LoRa v4.0

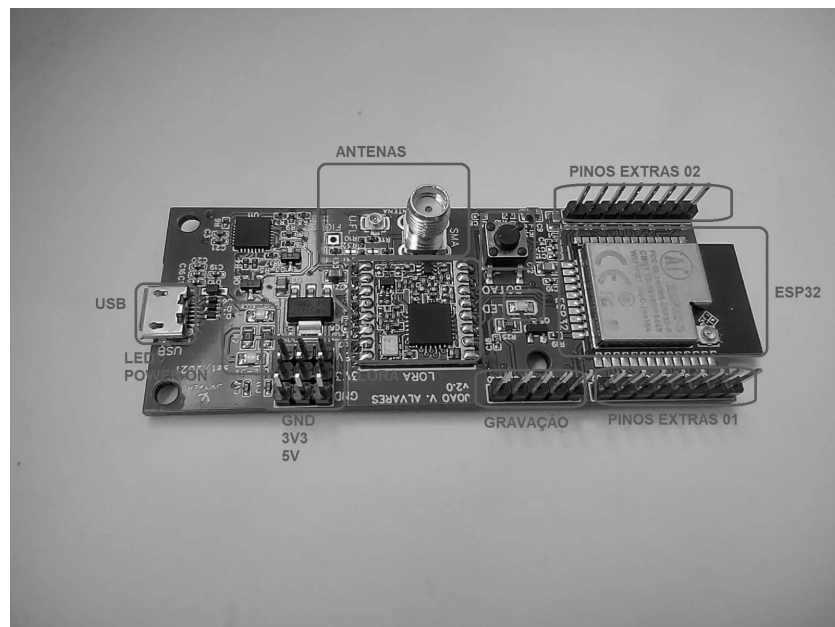


FIGURA 2 – Placa ESP32 LoRa v2.0

## ▲ Comparação

A tabela a seguir apresenta a correspondência dos pinos marcados na placa com os pinos do ESP32:

| Pino        | Placa ESP32 LoRa v1.0 |
|-------------|-----------------------|
| GND (Terra) | GND                   |
| GPIO0       | 23                    |
| EMAC TXER   |                       |
|             |                       |

|             |    |  |
|-------------|----|--|
| ADC2_1      |    |  |
| RTCIO 11    |    |  |
| Touch 1     |    |  |
| Clock Out 1 | 22 |  |
| GPIO2       |    |  |
| Hspi WP     |    |  |
| ADC2_2      |    |  |
| RTCIO 12    |    |  |
| Touch 2     |    |  |
| GPIO15      | 21 |  |
| EMAC RXD3   |    |  |
| ADC2_3      |    |  |
| RTCIO 13    |    |  |
| Touch 3     |    |  |
| GPIO14      | 17 |  |
| EMAC TXD2   |    |  |
|             |    |  |



|           |    |  |
|-----------|----|--|
| ADC2_6    |    |  |
| RTCIO 16  |    |  |
| Touch 6   |    |  |
| GPI027    | 16 |  |
| EMAC RXDV |    |  |
| ADC2_7    |    |  |
| RTCIO 17  |    |  |
| Touch 7   | 04 |  |
|           |    |  |
| GPI026    | 15 |  |
| EMAC RXD1 |    |  |
| ADC2_9    |    |  |
| RTCIO 7   |    |  |
| DAC 2     |    |  |
| GPI023    | 36 |  |
| GPI022    | 39 |  |
|           |    |  |

|            |    |
|------------|----|
| EMAC TXD1  | 34 |
| GPIO5      |    |
| EMAC RXCLK |    |
| GPIO18     | 35 |
| GPIO7      | 32 |
| SPIQ       |    |
| GPIO8      | 33 |
| SPID       |    |
| GPIO32     | 12 |
| ADC2_4     |    |
| RTCIO 9    |    |
| Touch 9    |    |
| XTAL 32    |    |
| GPIO33     | 13 |
| ADC2_5     |    |
| RTCIO 8    |    |

|         |  |
|---------|--|
| Touch 8 |  |
| XTAL 32 |  |

## ▲ Programação

Para realizar a programação do ESP32, é possível utilizar a plataforma Arduino IDE, apenas adicionando algumas bibliotecas próprias. Dessa forma, primeiramente deve-se abrir a aba “Arquivo” localizada na parte superior do aplicativo. Dentro dessa aba é necessário abrir o menu denominado “preferências”. Também é possível abrir esse menu utilizando o atalho do programa: CTRL + , (control + vírgula).

Ao entrar no menu “preferências” uma aba sobreposta aparecerá com diversas propriedades alteráveis do programa. Desde o tamanho da fonte, o idioma da plataforma, o tema utilizado, entre outras. Na parte final há uma configuração denominada “URLs Adicionais para Gerenciadores de Placas” que permite adicionar links externos que possibilitam a configuração de placas além das pertencentes ao Arduino. Para conseguirmos configurar o módulo ESP32 é necessário adicionar um link após os dois pontos, esse URL é o seguinte: [https://dl.espressif.com/dl/package\\_esp32\\_index.json](https://dl.espressif.com/dl/package_esp32_index.json)

Na Figura 2 a seguir é possível ver a configuração que deve ser feita nessa página.

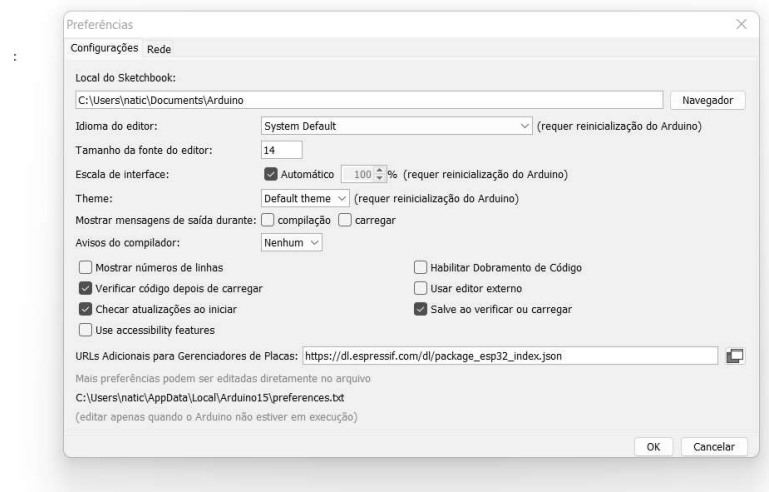


FIGURA 3 – Configuração em Preferências

Após isso, é necessário baixar as ferramentas necessárias. Para isso iremos primeiramente na aba Ferramentas, então dentro da aba Placas escolher a opção “Gerenciador de placas”. No menu que abrirá, em sua parte superior há um espaço de pesquisa. Nessa linha basta escrever “ESP32” e a única biblioteca de ferramentas que aparecer é a que deve ser baixada. Basta escolher a versão mais recente e instalá-la. Na Figura 3 a seguir essa aba de gerenciadora de placas e a ferramenta do ESP32 podem ser vistas.

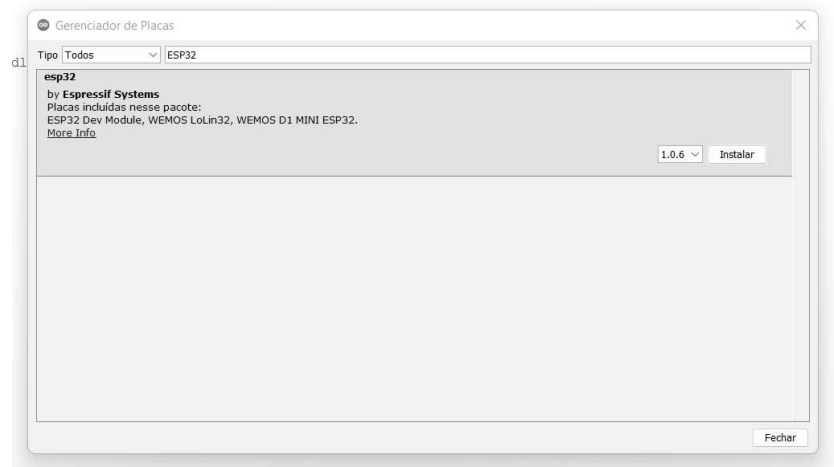


FIGURA 4 – Configuração em Gerenciador de Placas

Outra instalação necessária é a biblioteca LoRa. Para isso entre na aba Ferramentas, seguindo para a aba Gerenciador de Bibliotecas. Essa aba também pode ser acessada pelo atalho CTRL + SHIFT + I (control + shift + I). Dentro dessa aba, na parte superior, é necessário digitar LoRa. A própria empresa da Arduino possui algumas bibliotecas relacionadas a tecnologia LoRa, para programar é necessário instalar as seguintes bibliotecas: LoRa (by Sandeep Mistry) e Esplora (by Arduino). Novamente, é apenas necessário selecionar a versão mais recente e instalá-la. Na Figura 4 a seguir isso é visto.

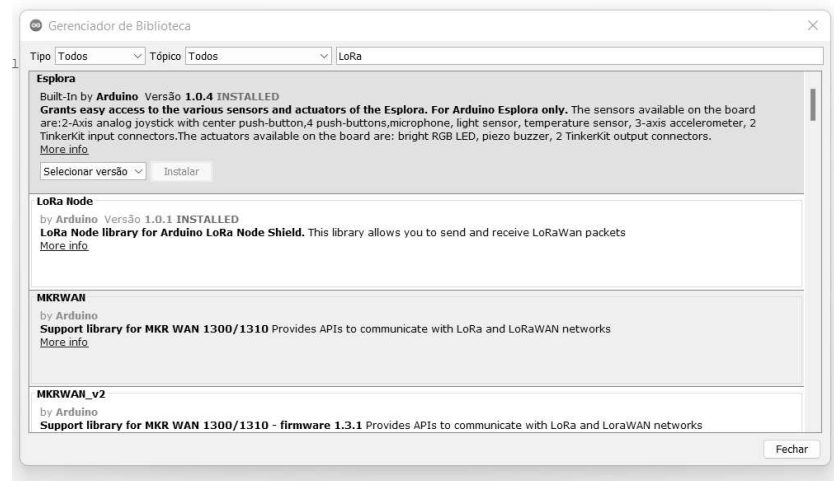


FIGURA 5 – Configuração em Gerenciador de Bibliotecas

Com todas as ferramentas e bibliotecas instaladas, é necessário configurar qual placa ESP32 estamos utilizando, configurando também o LoRa. Para isso navegamos até a aba Ferramentas, Placa e ESP32 Arduino. Dentro desse menu, há uma longa lista de opções, sendo a correta a denominada “AI Thinker ESP32-CAM” como pode ser visto na Figura 5 a seguir.

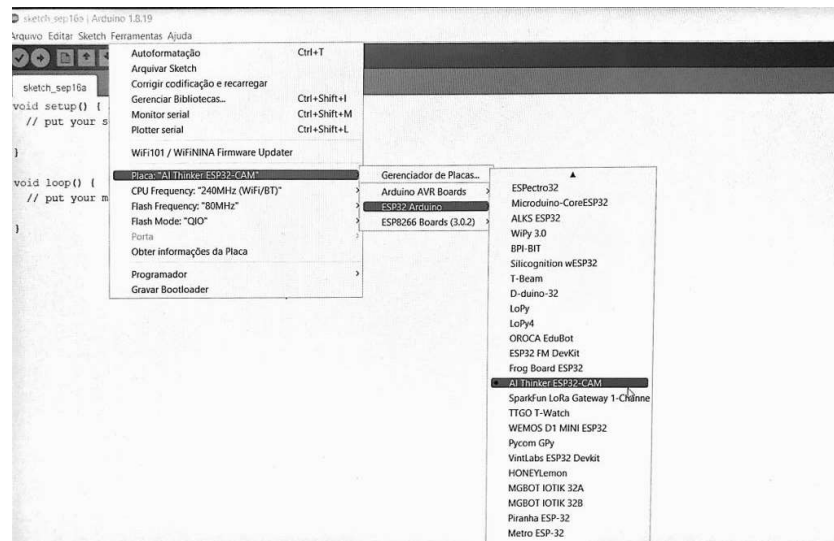


FIGURA 6 – Configuração de Placa

## ▲ Exemplo

### Transmissão entre Placas

Para a realização desse exemplo programaremos as duas versões da placa ESP32 LoRa, sendo que ambas irão desempenhar os papéis de emissoras e receptoras dependendo das circunstâncias. Para isso é necessário fazer uma versão do código específico para cada placa, apesar de ambas terem diversas similaridades no código.

Transmitiremos uma simples mensagem numerada que será verificada e marcada como correta.

Para a placa emissora, será utilizado a quarta versão, contudo também é possível utilizar a outra versão. Primeiramente incluímos as duas bibliotecas necessárias para

a comunicação com o LoRa e o funcionamento ideal. Então definimos os pinos NSS, RESET E DPIO0 do LORA em relação ao ESP32. Considerando a placa ESP32 LoRa JVTech nós temos que esses pinos são: NSS 30, RESET 13, DPIO0 11. Além disso, iniciamos uma variável global chamada ciclo para numerar as mensagens enviadas.

Dentro da função *setup* primeiramente iniciaremos a comunicação serial, definindo sua velocidade como 115200. A mensagem “Lora Emissor” será emitida sinalizando que tudo está funcionando até agora. Então configuramos os pinos do Lora como previamente definidos com a função adequada.

Seguindo, há um comando repetitivo que mostrará três pontos enquanto o LoRa é iniciado e parará quando o LoRa receptor for devidamente iniciado. Finalmente, temos a sincronização com o LoRa receptor utilizando uma palavra sincronizadora. Essa varia entre 0 e 0xFF, podendo ser qualquer valor desde que seja a mesma para ambos o receptor e o emissor. Há também uma mensagem que sinaliza o funcionamento correto da sincronização.

Continuando para a função *loop*, temos primeiro a função da placa como emissora. Inicialmente, pela comunicação serial, essa placa avisará que está transmitindo e piscará o LED uma vez como indicador. Então começamos o envio da mensagem. Isso é feito por meio de um pacote que deverá ser aberto e fechado por funções presentes nas bibliotecas do LoRa. Dessa forma, tudo dentro das funções de



pacote será transmitido como mensagem. A placa também funcionará como receptora por meio da função de percepção de pacote. Essa placa somente piscará o LED e mostrará na comunicação serial quando receber uma mensagem, caso contrário ela continuará a agir como emissora. Para receber a mensagem primeiro é constatado se há um pacote com uma função condicional. Então o recebimento será avisado na comunicação serial e o LED piscará. Em seguida a função específica ligado ao LoRa lerá a mensagem. Por fim temos o acréscimo da constante ciclo que está associada a função condicional, significando que a mensagem só mudará quando a outra placa comunicar que a recebeu. Há também um *delay* de dois segundos para melhorar o funcionamento do sistema. Nas Figuras 7 e 8 a seguir há o código dessa primeira placa.

```

//Bibliotecas
#include <SPI.h>
#include <LoRa.h>

//Definição dos pinos utilizados pelo LoRa
#define SS 30
#define RST 13
#define D0 11

int ciclo = 0;

void setup() {
  //Inicia comunicação serial
  Serial.begin(115200);

  //Configura o botão e o LED
  pinMode(10, INPUT); //Botão
  pinMode(2, OUTPUT); //LED

  //Configura os pinos do Lora
  LoRa.setPins(SS, RST, D0);

  //Configurando a frequência do LoRa
  while (!LoRa.begin(915E6)) {
    Serial.println(" ... ");
    delay(500);
  }

  //Sincroniza LoRa Emissor-Receptor
  LoRa.setSyncWord(0xAA);
  Serial.println("Sincronização Completa");
}

```

FIGURA 7 – Parte 1 do código da placa v4.0

```

void loop() {
    //Placa é Emissor
    Serial.print("Emitindo Mensagem: ");

    //Pisca o LED
    digitalWrite(2, HIGH);
    delay(1000);
    digitalWrite(2, LOW);

    //Enviando a mensagem
    LoRa.beginPacket();
    LoRa.print(ciclo);
    LoRa.endPacket();
    delay(2000);

    //Placa é Receptor
    int packetSize = LoRa.parsePacket();
    if (packetSize)
    {
        //Uma mensagem foi recebida
        Serial.print("Mensagem Recebida");

        //Pisca o LED
        digitalWrite(2, HIGH);
        delay(1000);
        digitalWrite(2, LOW);

        //Leitura da mensagem
        while (LoRa.available())
        {
            String LoRaData = LoRa.readString();
            Serial.print(LoRaData);
        }

        ciclo++; //Aumenta após recebimento da confirmação
    }

    delay (2000);
}
}

```

FIGURA 8 – Parte 2 do código da placa v4.0

Para a placa receptora, será utilizado a segunda versão, contudo também é possível utilizar a outra versão. Primeiramente incluímos as duas bibliotecas necessárias para a comunicação com o LoRa e o funcionamento ideal. Então definimos os pinos NSS, RESET E DPIO0 do LORA em relação ao ESP32. Considerando a placa ESP32 LoRa JVTech nós temos que esses pinos são: NSS 30, RESET 13, DPIO0 11. Além disso, iniciamos uma variável global chamada ciclo para numerar as mensagens enviadas e outra chamada mensagem para verificarmos a conexão entre placas.

Dentro da função *setup* primeiramente iniciaremos a comunicação serial, definindo sua velocidade como 115200. A mensagem “Lora Receptor” será emitida sinalizando que tudo está funcionando até agora. Então configuramos os pinos do Lora como previamente definidos com a função adequada.

Seguindo, há um comando repetitivo que mostrará três pontos enquanto o LoRa é iniciado e parará quando o LoRa receptor for devidamente iniciado. Finalmente, temos a sincronização com o LoRa emissor utilizando uma palavra sincronizadora que deve ser a mesma anteriormente configurada. Há também uma mensagem que sinaliza o funcionamento correto da sincronização.

Continuando para a função *loop*, temos primeiro a análise de transmissões. Quando uma mensagem é detectada, é aberta uma função condicional. Primeiramente há uma mensagem avisando que a transmissão foi recebida, então o

recebimento é avisado pela comunicação serial e o LED da placa pisca uma vez. Após isso há a leitura da mensagem com as funções adequadas da tecnologia LoRa configurando a variável mensagem.

Após isso, há uma função condicional garantindo que a mensagem recebida é a mensagem esperada. Essa verificação é feita por meio da variável ciclo que está sincronizada nas duas placas para apenas ser alterada quando a mensagem anterior foi enviada, recebida, e reenviada verificada. Dentro dessa condicional, o módulo funcionará como emissor, avisando a comunicação serial, piscando o LED e mandando a mesma mensagem que recebeu com um “OK” adicional. Apenas quando essa verificação foi feita é que a variável ciclo será alterada. Caso a mensagem seja diferente da esperada o LED irá ficar ligado por cinco segundos para indicar o problema. Por fim, um *delay* garantirá uma execução tranquila. Nas Figuras 9, 10 e 11 a seguir teremos o código dessa placa.

```

//Bibliotecas
#include <SPI.h>
#include <LoRa.h>

//Definição dos pinos utilizados pelo LoRa
#define SS 30
#define RST 13
#define D0 11

int ciclo = 0;
int mensagem = 0;

void setup() {
  //Inicia comunicação serial
  Serial.begin(115200);

  //Configura o botão e o LED
  pinMode(10, INPUT); //Botão
  pinMode(2, OUTPUT); //LED

  //Configura os pinos do Lora
  LoRa.setPins(SS, RST, D0);

  //Configurando a frequência do LoRa
  while (!LoRa.begin(915E6)) {
    Serial.println(" ... ");
    delay(500);
  }

  //Sincroniza LoRa Emissor-Receptor
  LoRa.setSyncWord(0xAA);
  Serial.println("Sincronização Completa");
}

```

FIGURA 9 – Parte 1 do código da placa v2.0

```

void loop() {

    //Placa é Receptor
    int packetSize = LoRa.parsePacket();
    if (packetSize)
    {
        //Uma mensagem foi recebida
        Serial.print("Mensagem Recebida");

        //Pisca o LED
        digitalWrite(2, HIGH);
        delay(1000);
        digitalWrite(2, LOW);

        //Leitura da mensagem
        while (LoRa.available())
        {
            String LoRaData = LoRa.readString();
            mensagem = Serial.print(LoRaData);
        }
        delay(1000);
    }
}

```

FIGURA 10 – Parte 2 do código da placa v2.0

```

if (mensagem == Serial.print(ciclo))
{
    //Placa é Emissor
    Serial.print("Emitindo Mensagem: ");

    //Pisca o LED
    digitalWrite(2, HIGH);
    delay(1000);
    digitalWrite(2, LOW);

    //Enviando a mensagem
    LoRa.beginPacket();
    LoRa.print(ciclo);
    LoRa.print("OK");
    LoRa.endPacket();
    delay(2000);
    ciclo ++;
}
else
{
    //Pisca o LED
    digitalWrite(2, HIGH);
    delay(5000);
    digitalWrite(2, LOW);
}
delay (2000);
}

```

FIGURA 11 – Parte 3 do código da placa v2.0

Com tudo programado é apenas necessário configurar o hardware a ser utilizado. Para as duas versões da placa ESP32 Lora JVTech isso quer dizer conectar as antenas e carregar o código por meio do cabo micro-USB. Como ambas as versões funcionam com o conector mini SMA, esse será utilizado em ambos os casos. Além disso, é apenas necessário conectar os



dois módulos a um cabo micro-USB e carregar o código respectivo de cada placa. Nas Figuras 12 e 13 a seguir é possível visualizar a conexão de cada uma das placas na antena e no micro-USB em momentos em que cada placa está funcionando.

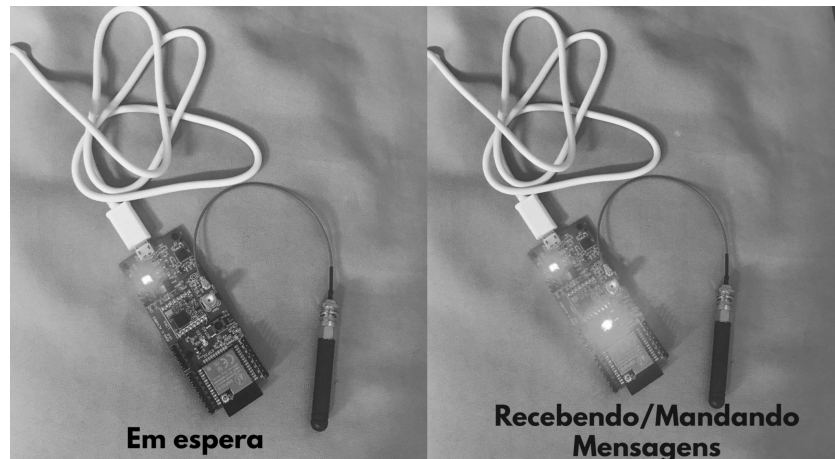


FIGURA 11 – Montagem da placa v2.0

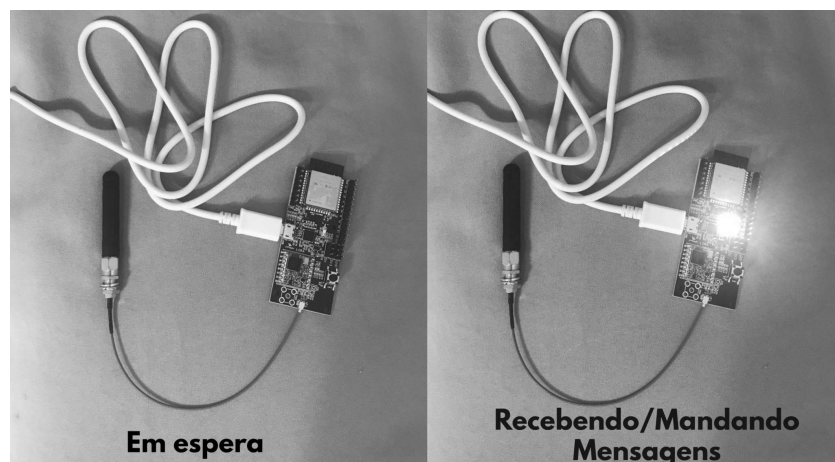


FIGURA 12 – Montagem da placa v4.0

[← Previous Artigo](#)

[Next Artigo →](#)

## Related Posts

Como utilizar a placa módulo Arduino Nano  
JVTechv 1.0

Tech

Como utilizar a Fonte de Bancada Fixa e Ajustável  
JVTech

Tech